



1. 图的存储结构

邻接矩阵法

邻接表法

十字链表

邻接多重表

1. 图的存储结构

邻接矩阵法

邻接表法

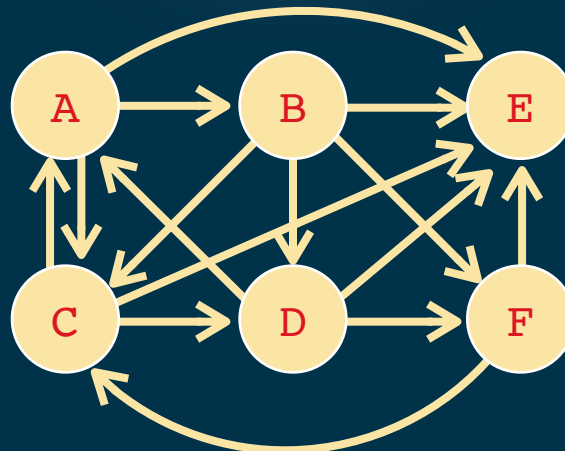
十字链表

邻接多重表

BOK数据结构



逻辑视角



数组视角

A[6][6]

	A	B	C	D	E	F
A	0	1	1	0	1	0
B	0	0	1	1	1	1
C	1	0	0	1	1	0
D	1	0	0	0	1	1
E	0	0	0	0	0	0
F	0	0	1	0	1	0



1. 图的存储结构

邻接矩阵法

邻接表法

十字链表

邻接多重表

```
#define MaxInt 32767 //表示极大值, 即 $\infty$ 
#define MVNum 100 //最大顶点数
typedef struct{
    VerTexType vexs[MVNum]; //顶点表
    ArcType arcs[MVNum][MVNum]; //邻接矩阵
    int vexnum, arcnum; //图的当前点数和边数
}AMGraph;
```



1. 图的存储结构

①初始化

邻接矩阵法

邻接表法

十字链表

邻接多重表

```
Status CreateUDN (AMGraph &G) {
    //采用邻接矩阵表示法, 创建无向网 G
    cin>>G.vexnum>>G.arcnum; //输入总顶点数, 总边数
    for (i=0;i<G.vexnum;++i) //依次输入点的信息
        cin>>G.vexs[i];
    for (i=0;i<G.vexnum; ++i) //初始化邻接矩阵, 边的权值均置为极大值MaxInt
        for(j=0; j<G.vexnum;++j)
            G.arcs[i][j]=MaxInt;
    for (k=0;k<G.arcnum;++k) //构造邻接矩阵
    {
        cin>>v1>>v2>>w; //输入一条边依附的顶点及权值
        i=LocateVex (G,v1) ; //确定v1和v2在G中的位置, 即顶点数组的下标
        j=LocateVex (G,v2) ; //同上
        G.arcs[i][j]=w; //边<v1, v2>的权值置为w
        G.arcs[j][i]=G.arcs[i][j]; //置<v1, v2>的对称边<v2, v1>的权值为w
    }
    return OK;
}
```



1. 图的存储结构

邻接矩阵法

邻接表法

十字链表

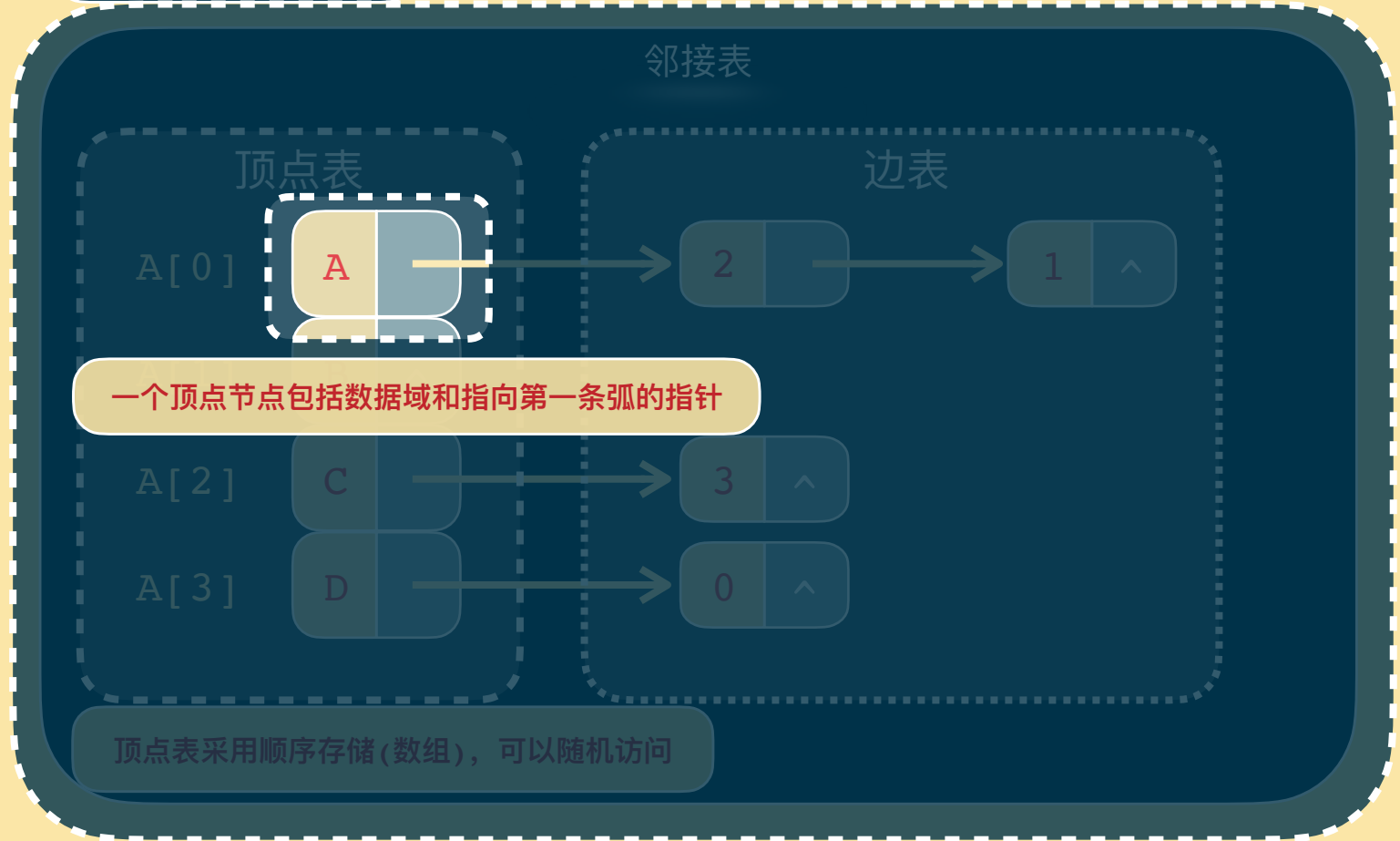
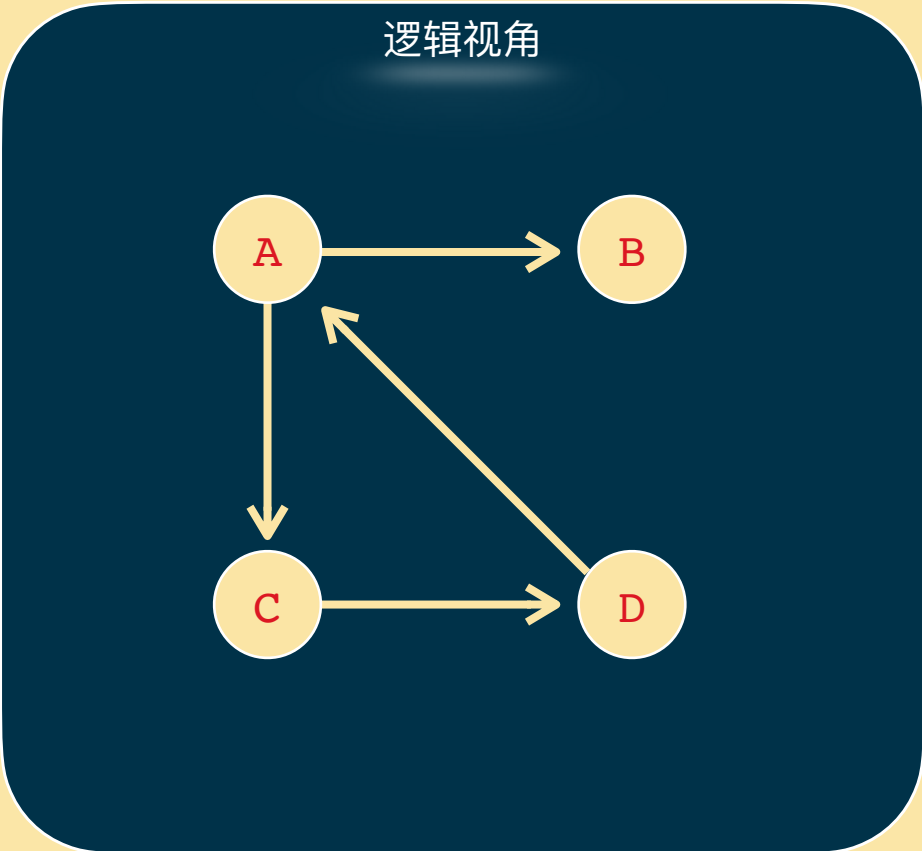
邻接多重表

优点	缺点
便于判断两个顶点间是否有边	不便增加和删除节点
便于计算各个顶点度	不便统计边的数目
	空间复杂度高



1. 图的存储结构

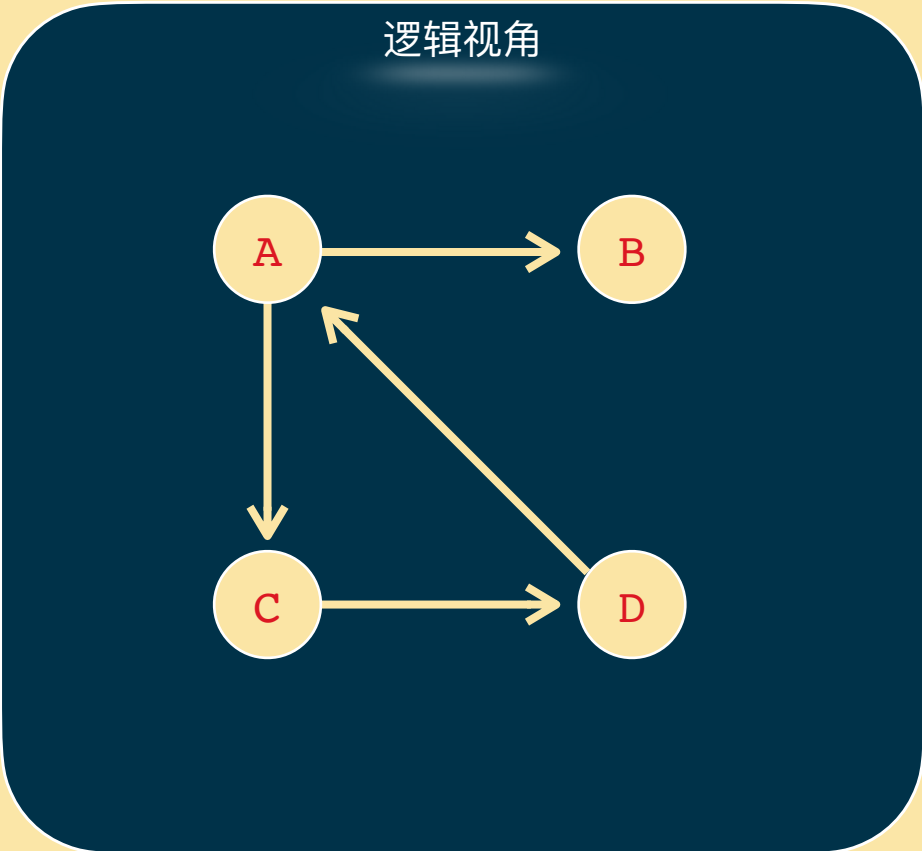
- 邻接矩阵法
- 邻接表法**
- 十字链表
- 邻接多重表





1. 图的存储结构

- 邻接矩阵法
- 邻接表法**
- 十字链表
- 邻接多重表

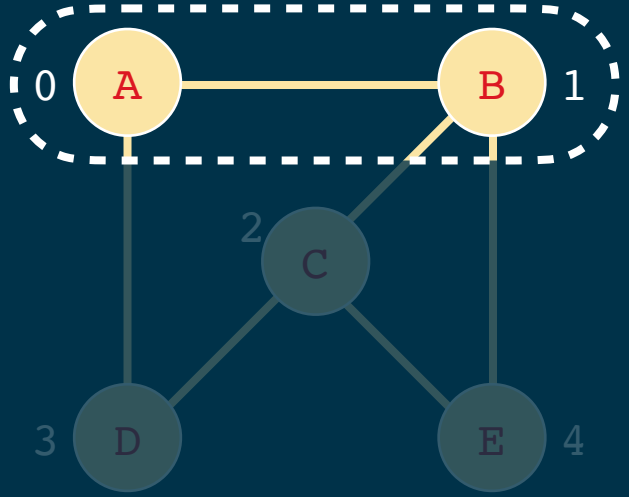




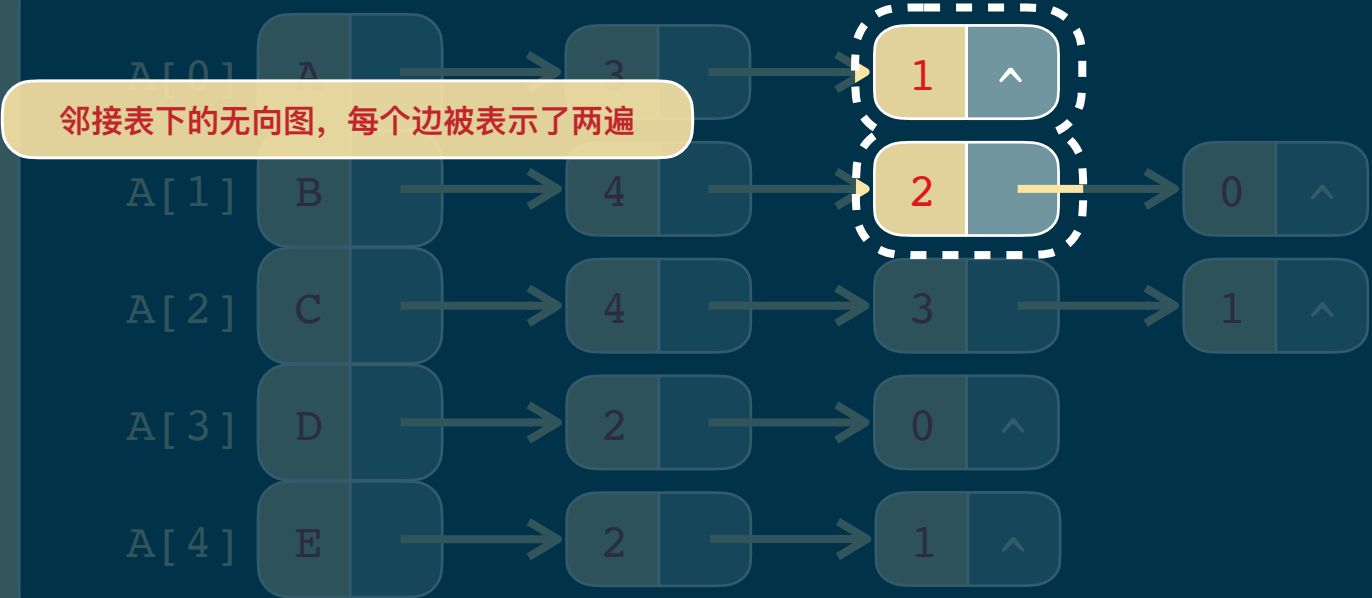
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表
- 邻接多重表

逻辑视角



邻接表



对于邻接表，如果G是无向图，那么存储空间需要 $O(|V|+2|E|)$ ，如果是有向图， $O(|V|+|E|)$ 并且邻接表的画法不唯一



1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表
- 邻接多重表

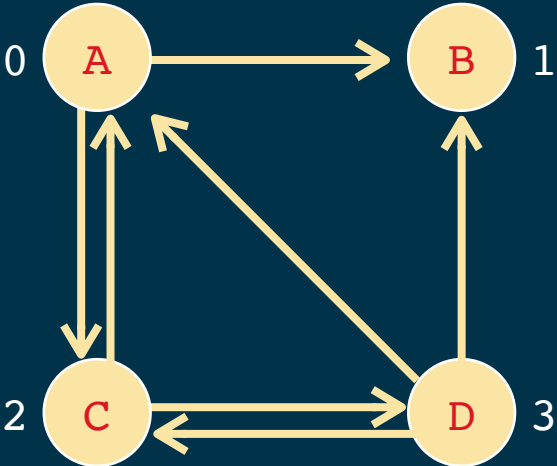
优点	缺点
便于增加删除节点	不便判断顶点之间是否有边
便于统计边的数目	不便计算有向图各个顶点的度
空间效率高	



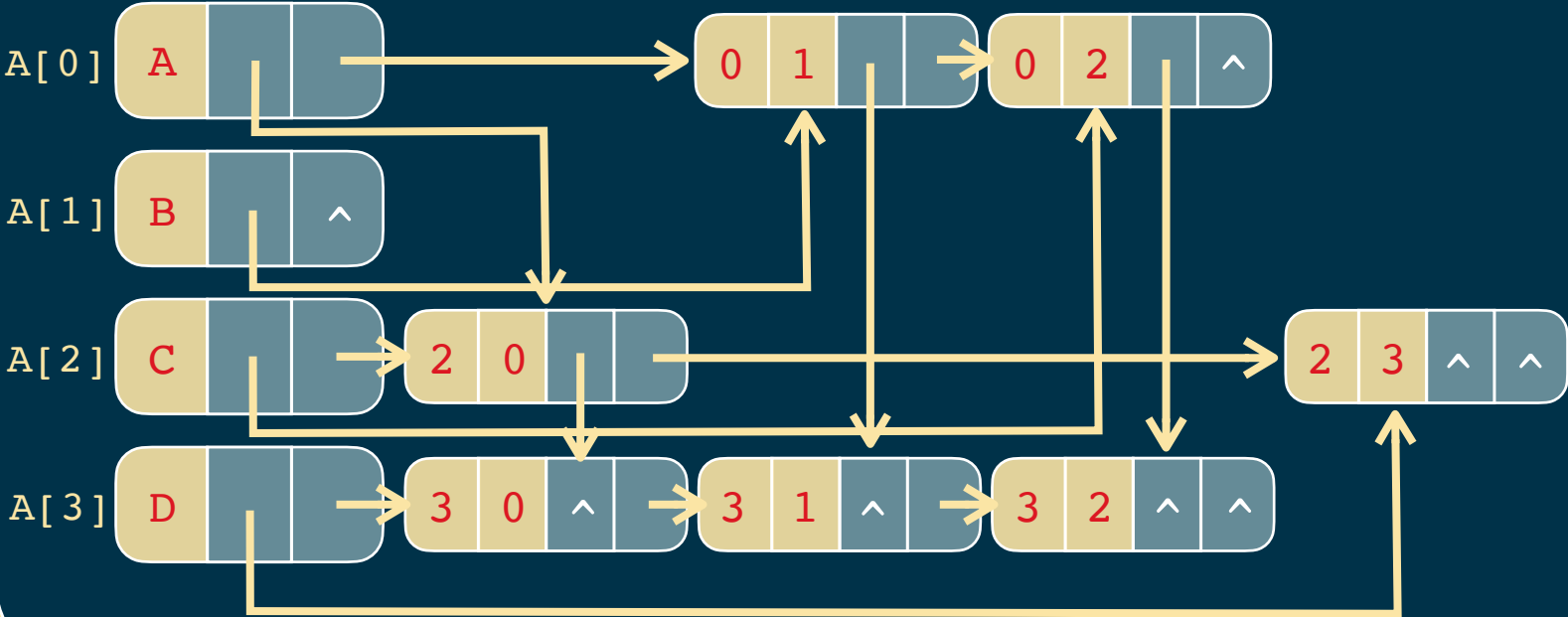
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

逻辑视角



十字链表

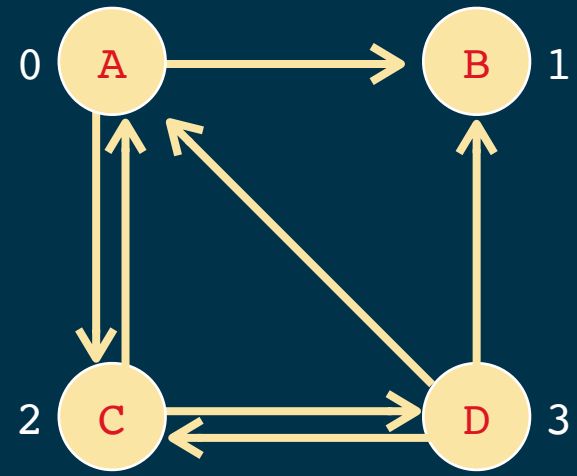




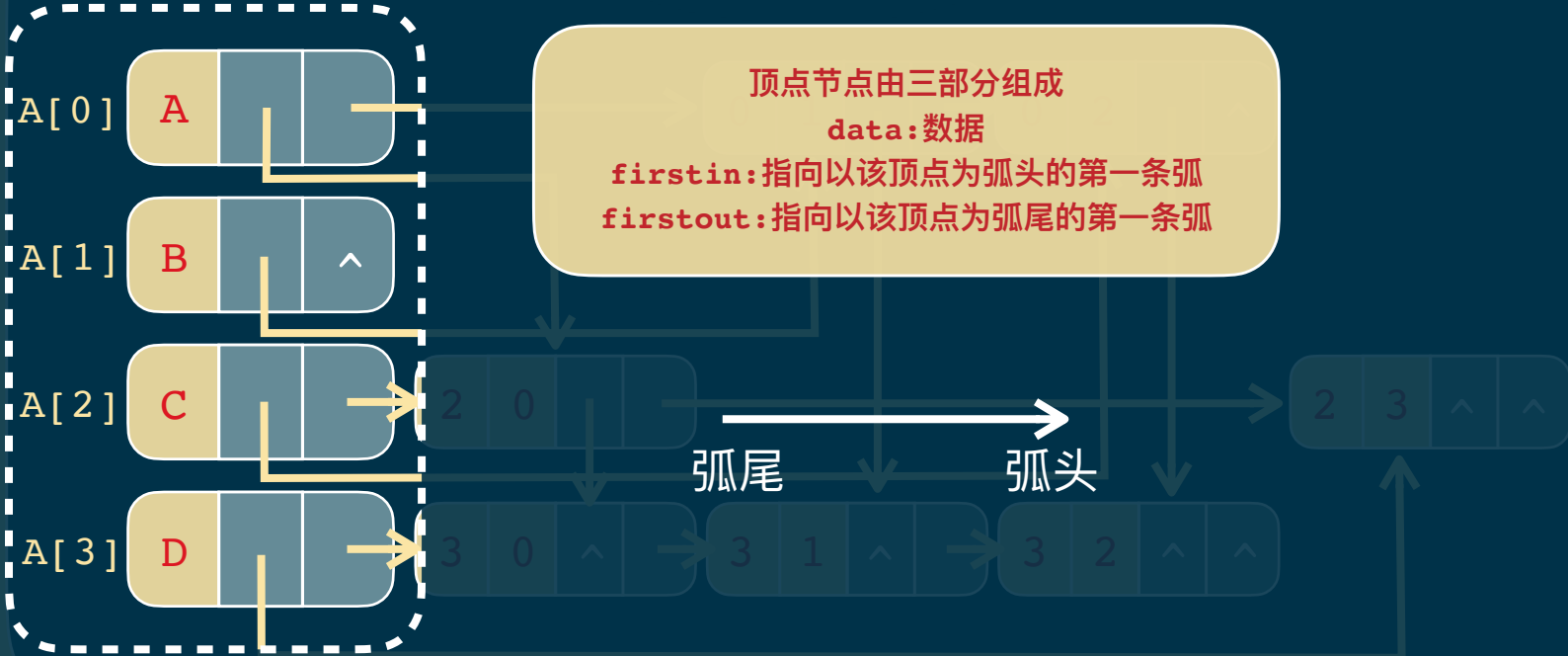
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

逻辑视角



十字链表

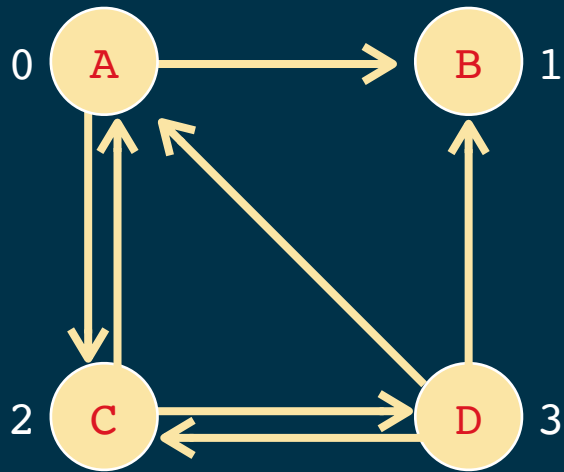




1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

逻辑视角



十字链表

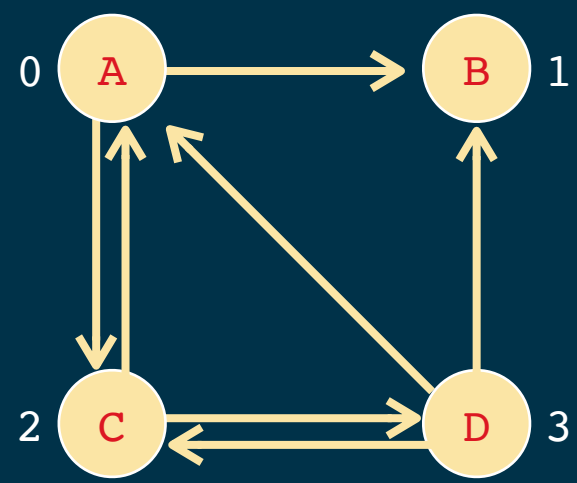




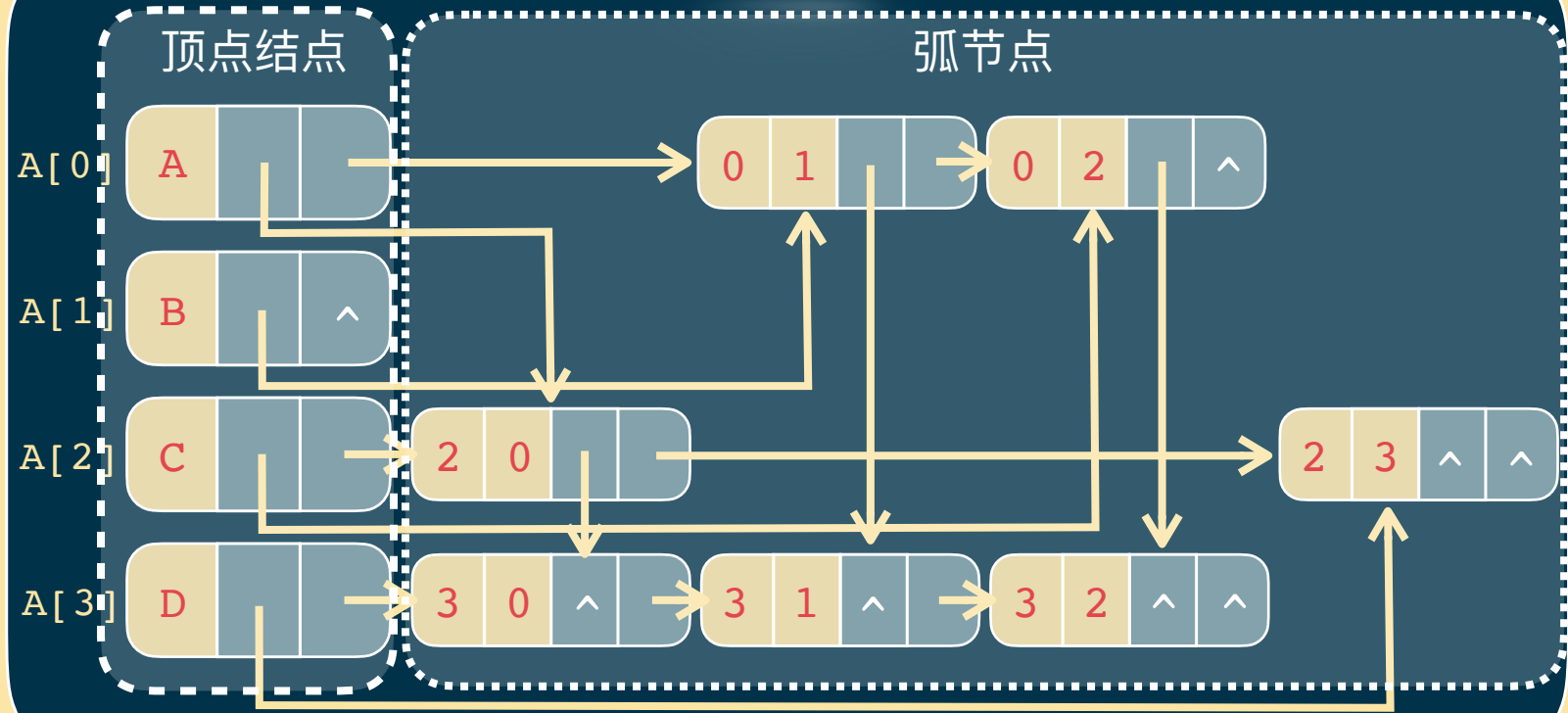
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

逻辑视角



十字链表





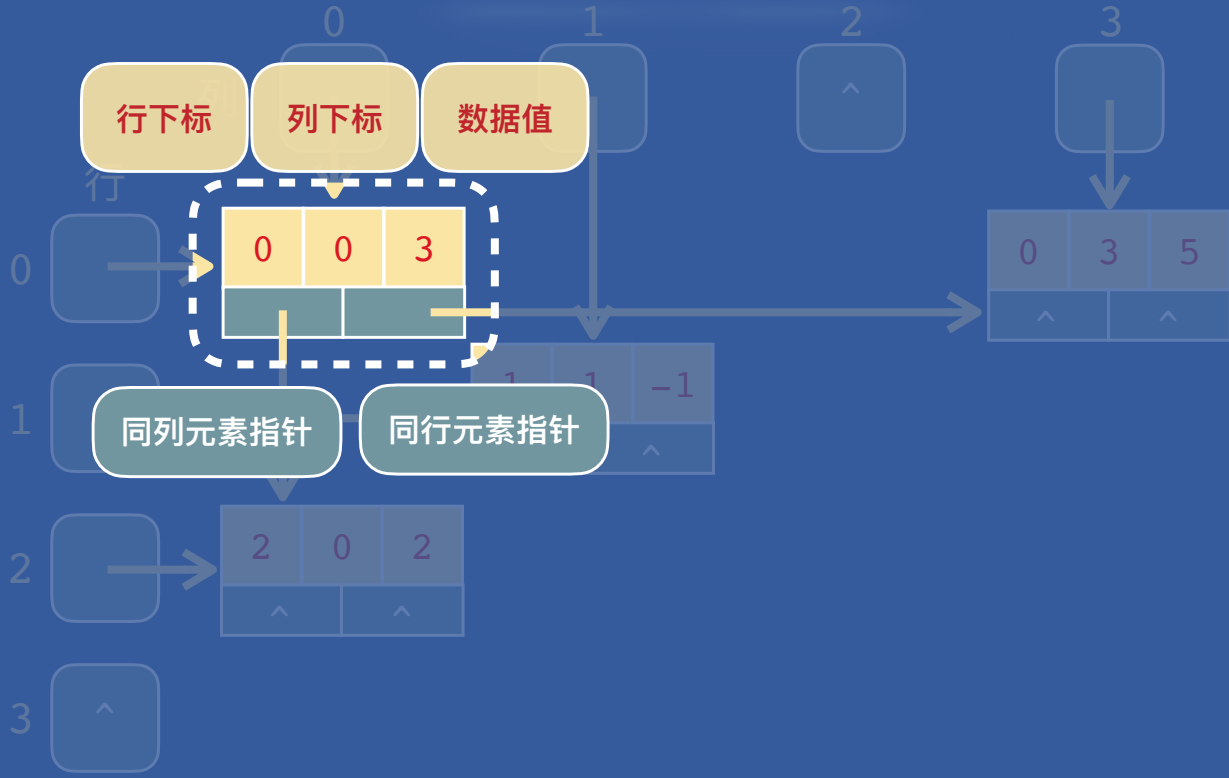
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

稀疏矩阵

3	0	0	5
0	-1	0	0
2	0	0	0
0	0	0	0

稀疏矩阵的十字链表表示



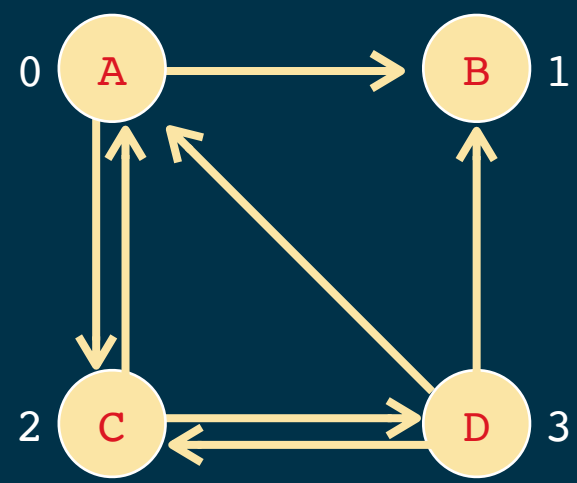


1. 图的存储结构

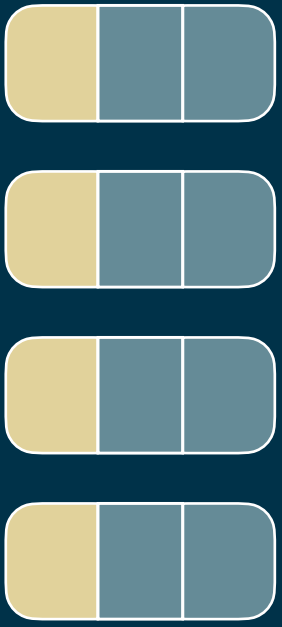
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

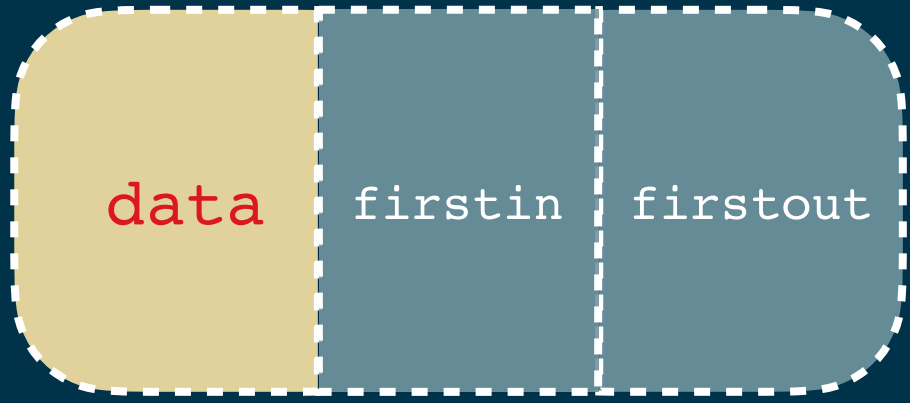
逻辑视角



先画出对应数量的节点



十字链表



节点结构

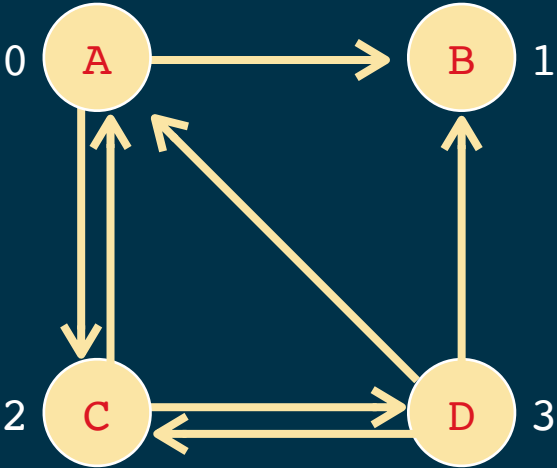


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表

填入数据

A		
B		
C		
D		

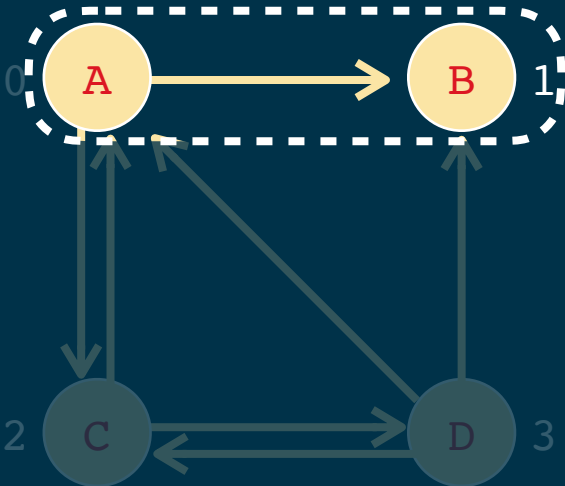


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

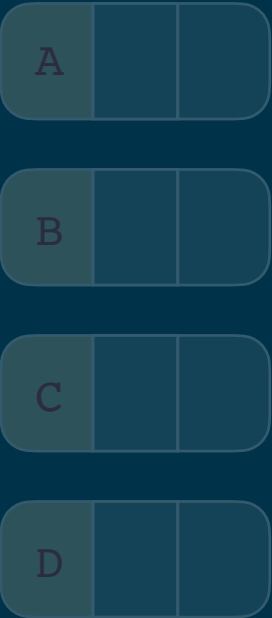
十字链表的画法

逻辑视角



接着逐个画边

十字链表



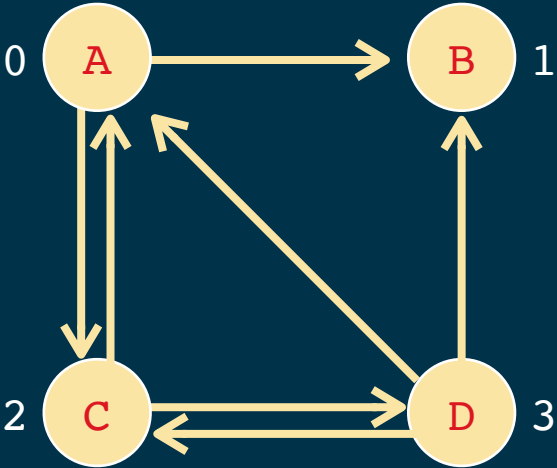


1. 图的存储结构

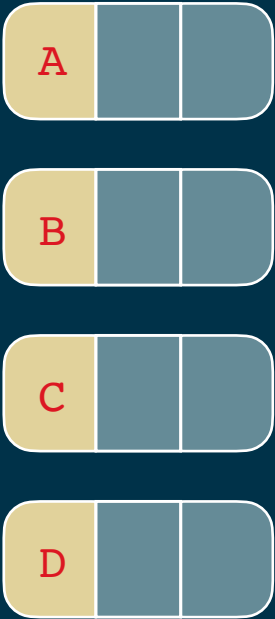
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表



tail vex	head vex	hlink	tlink
-------------	-------------	-------	-------

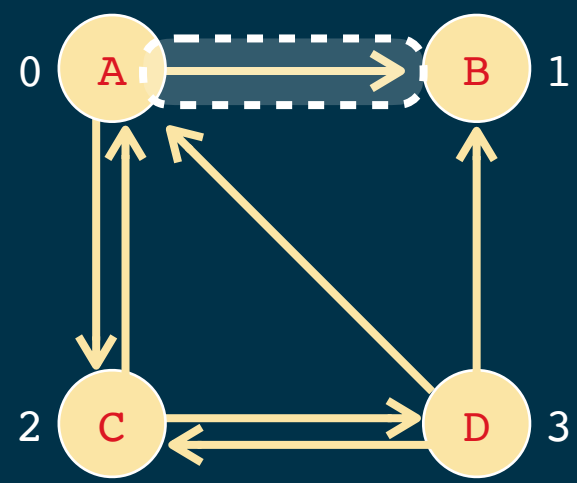


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表

A		
B		
C		
D		

0	1		
---	---	--	--

A(0)连接B(1)的边，在边表里填入对应数字

tail vex	head vex	hlink	tlink
-------------	-------------	-------	-------

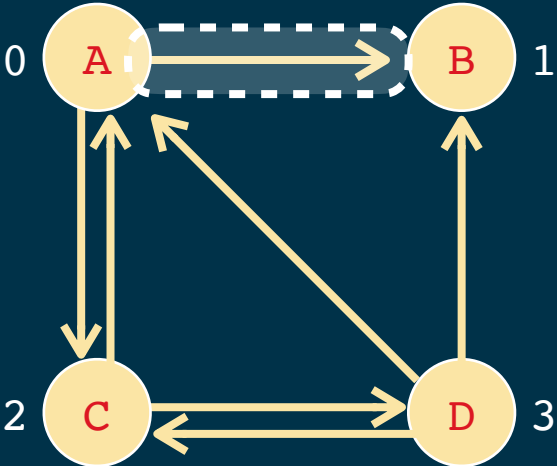


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



	0	1	2	3
A		0 1		
B				
C				
D				

十字链表的画法

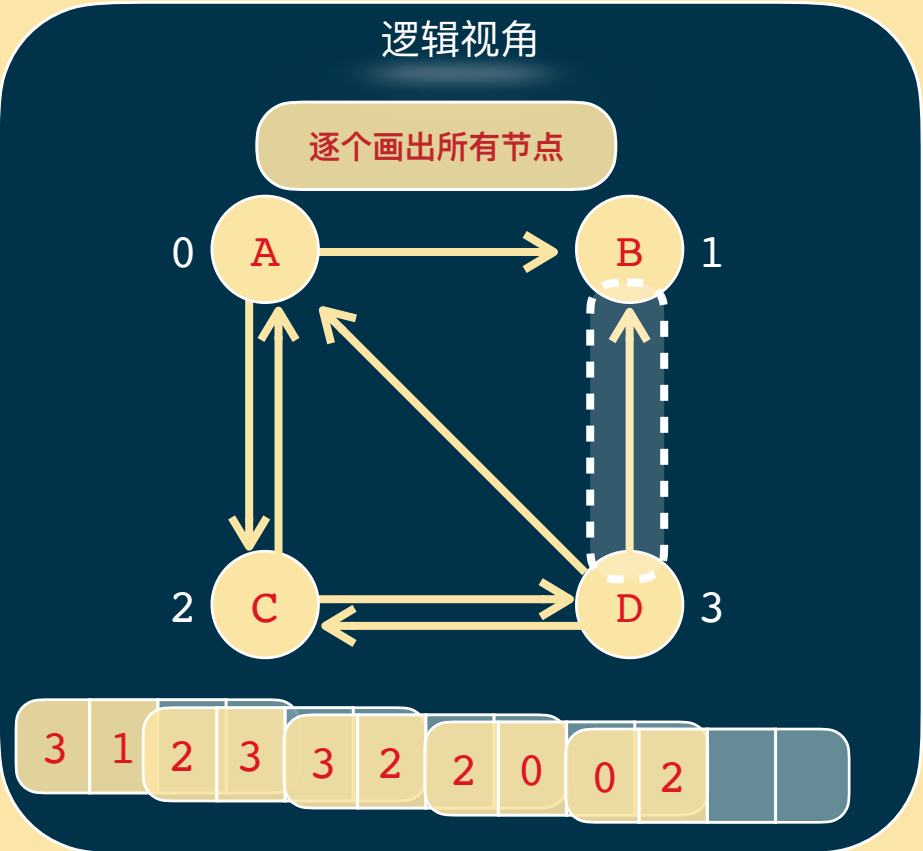
写出节点以后，两个顶点编号当做坐标，按照矩阵方式摆放好



1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法



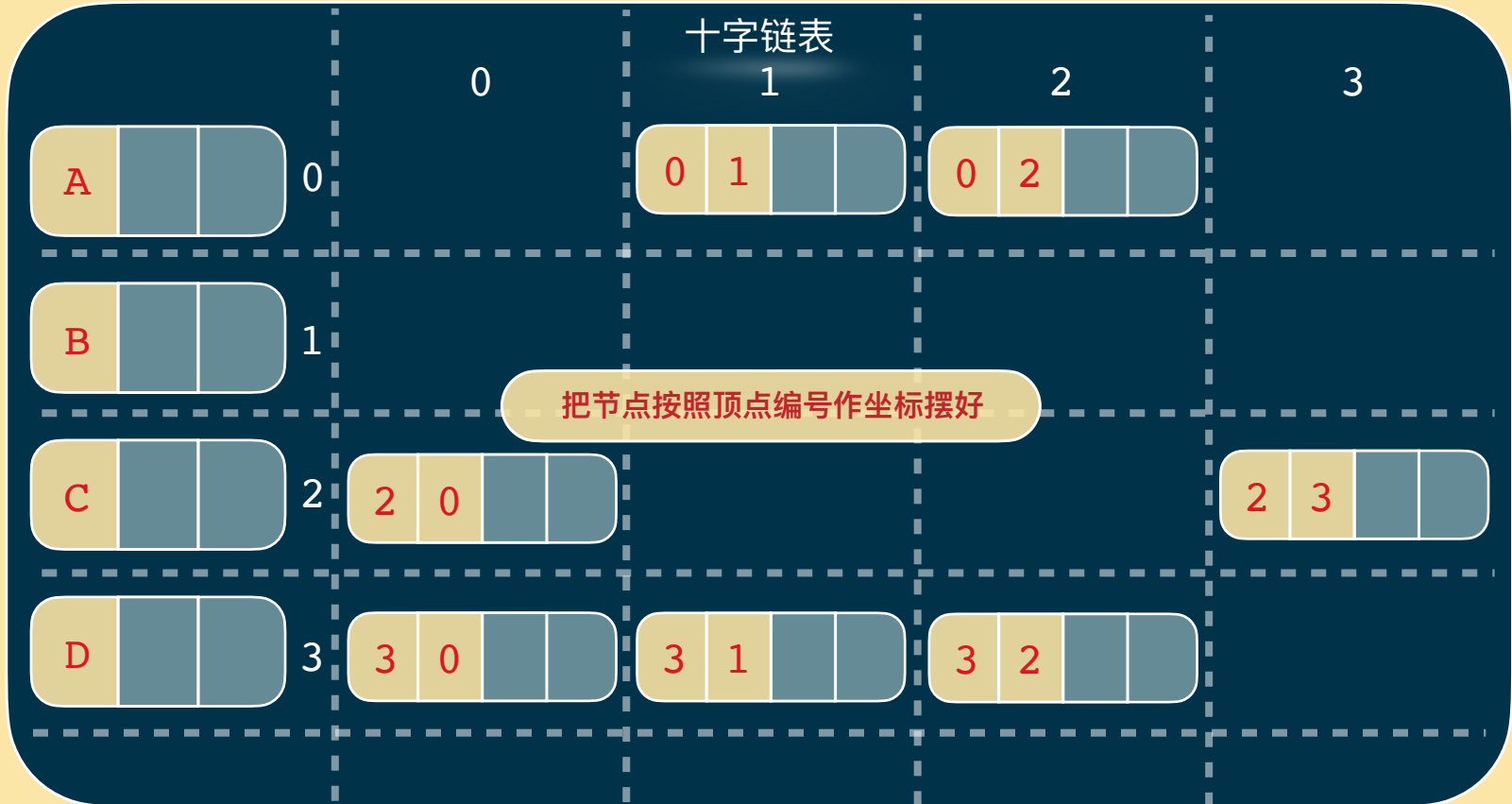
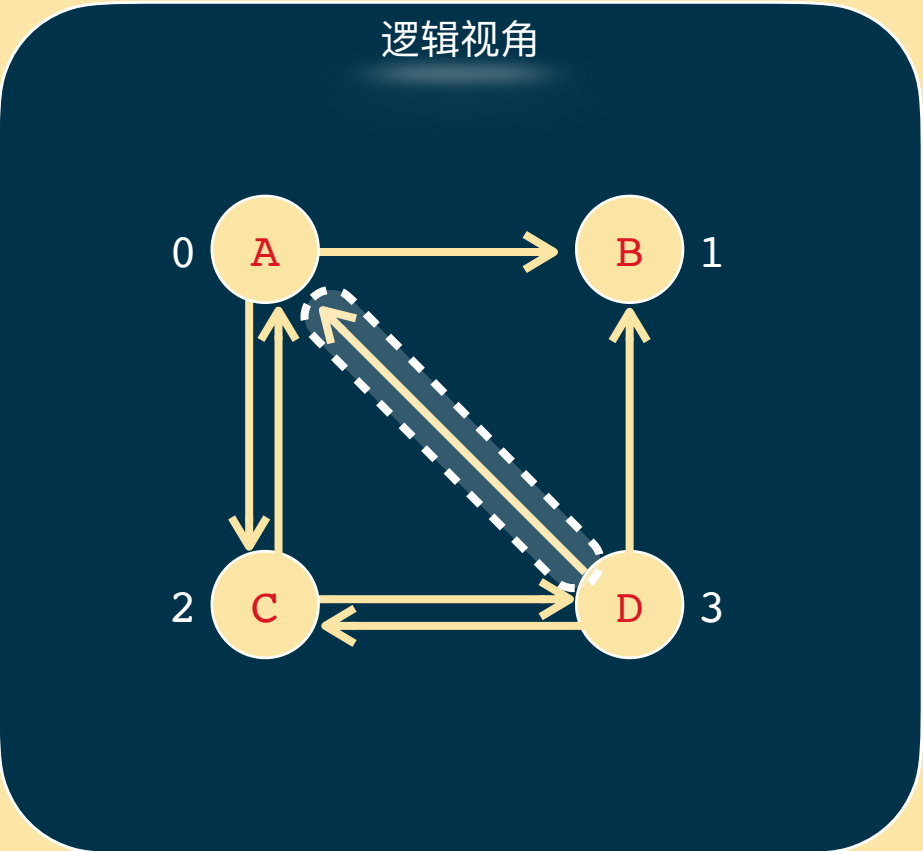
	0	1	2	3
A		0 1		
B				
C				
D				



1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法



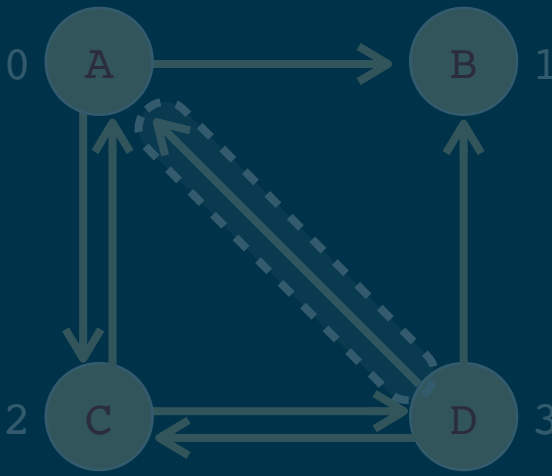


1. 图的存储结构

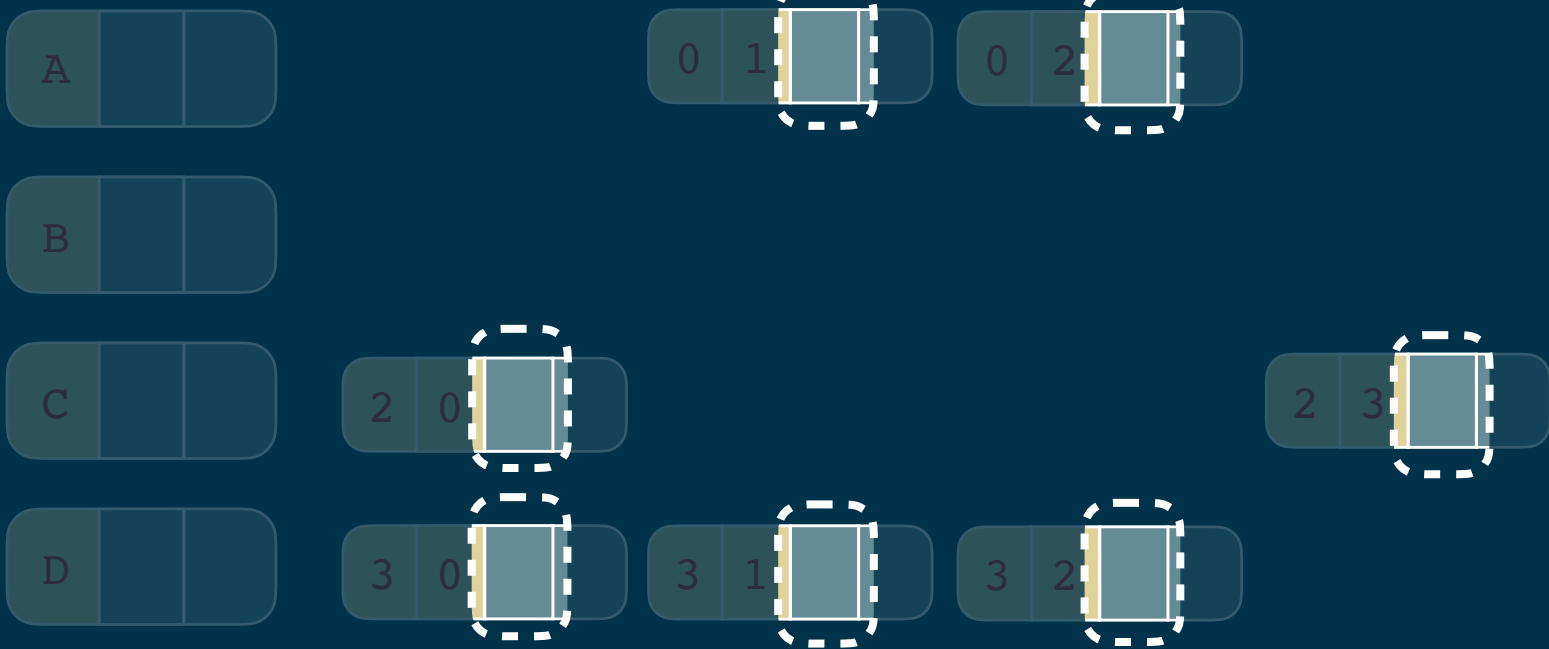
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



这里连好弧头相同的下一条弧



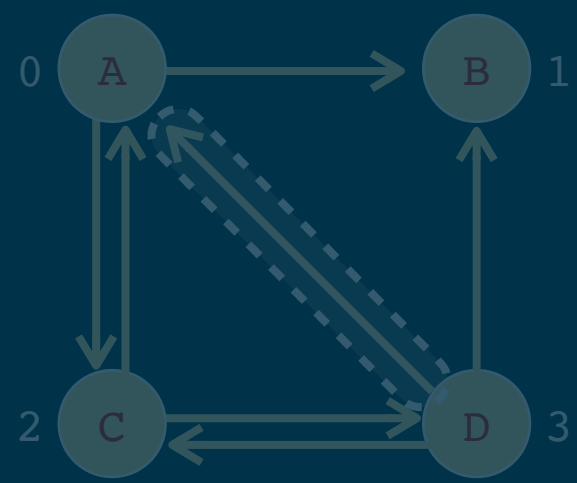


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



A		
B		
C		
D		



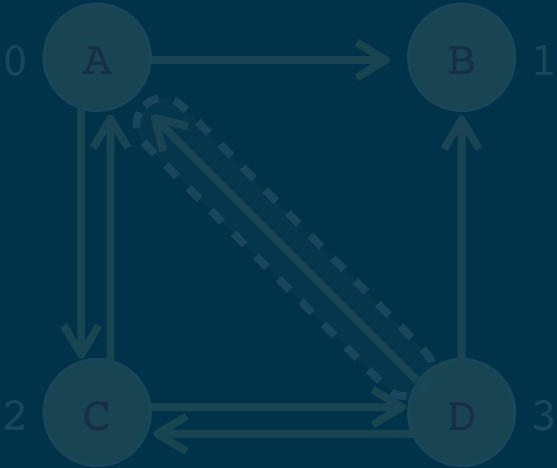


1. 图的存储结构

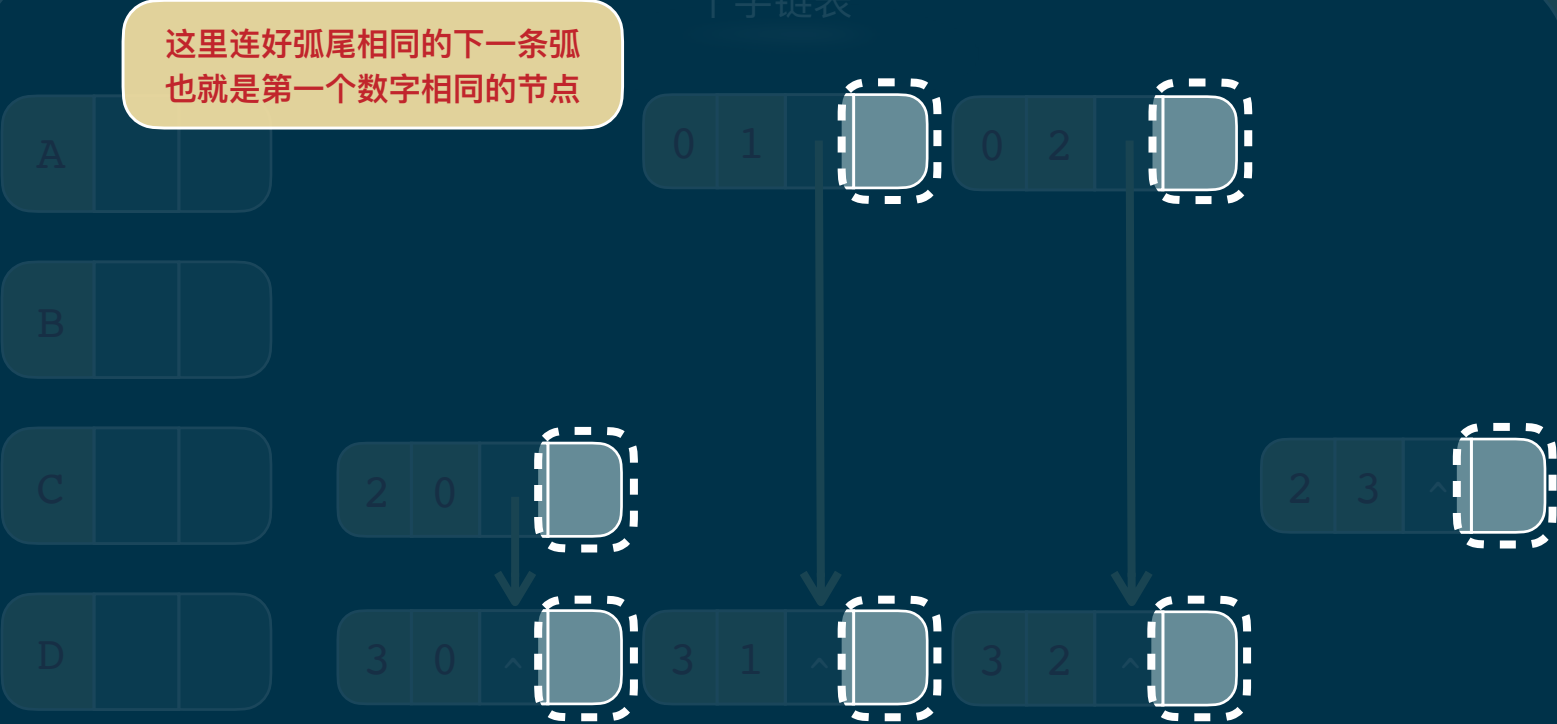
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表



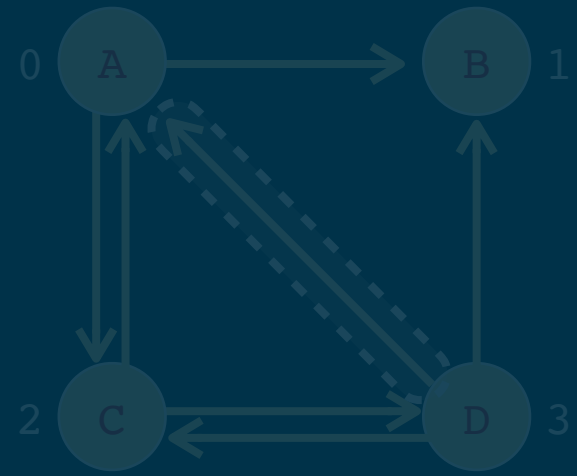


1. 图的存储结构

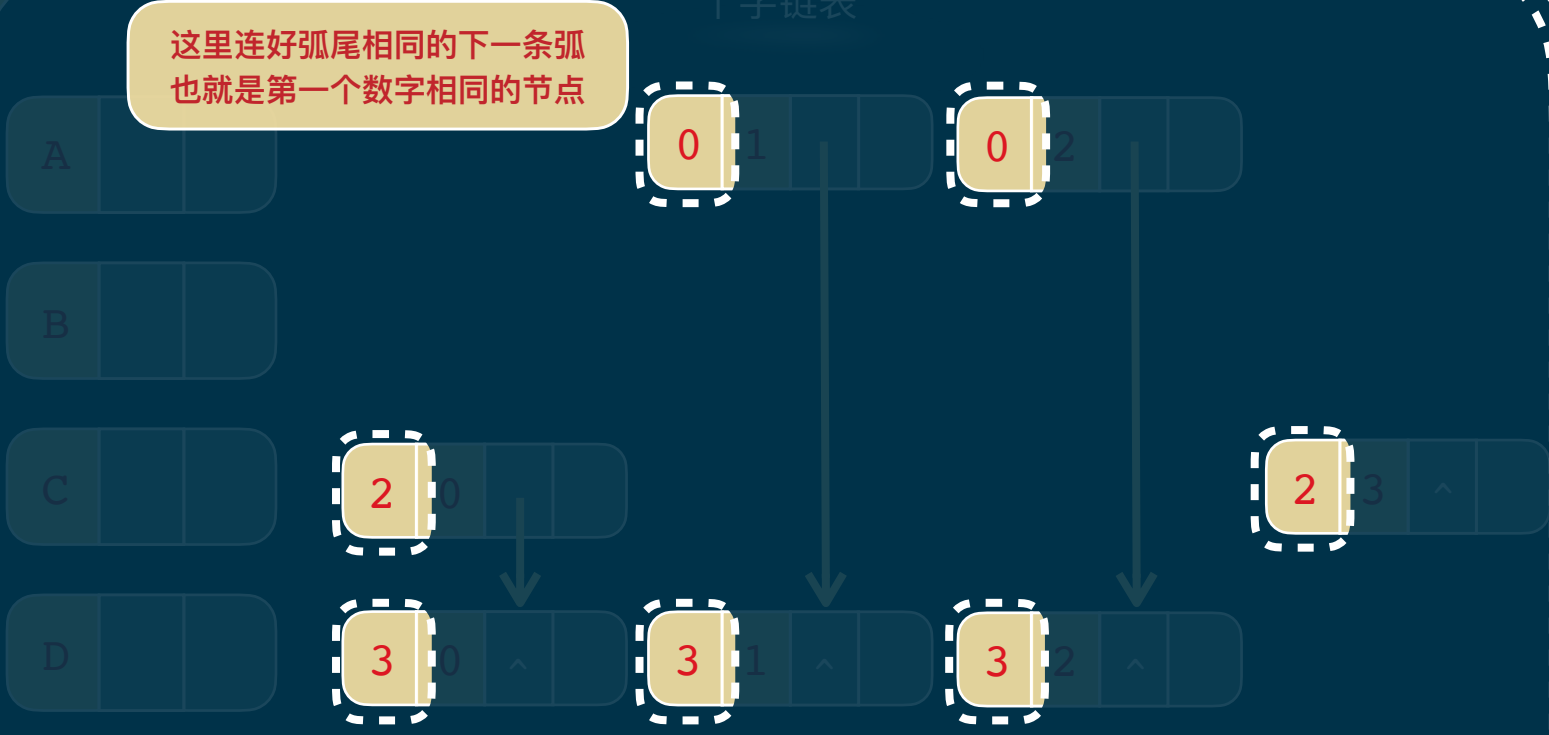
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表



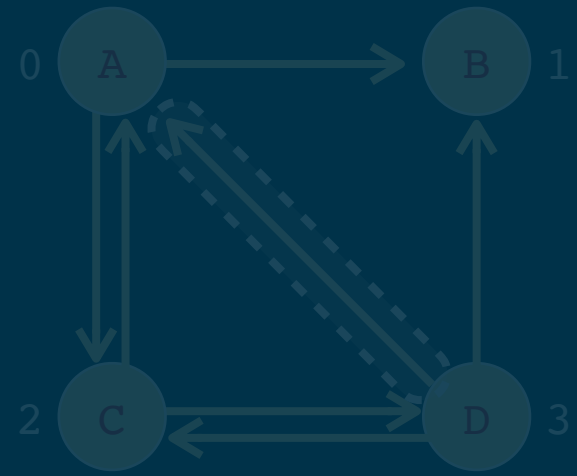


1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表



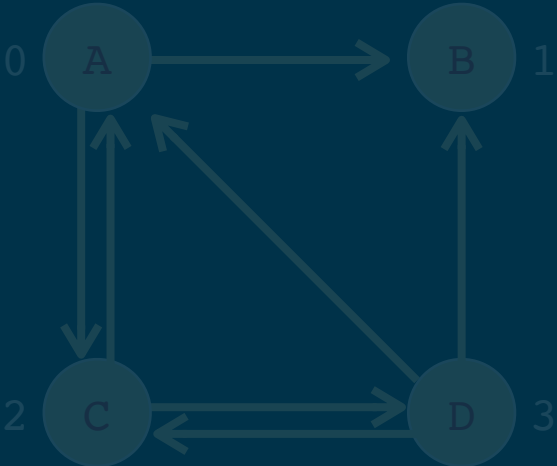


1. 图的存储结构

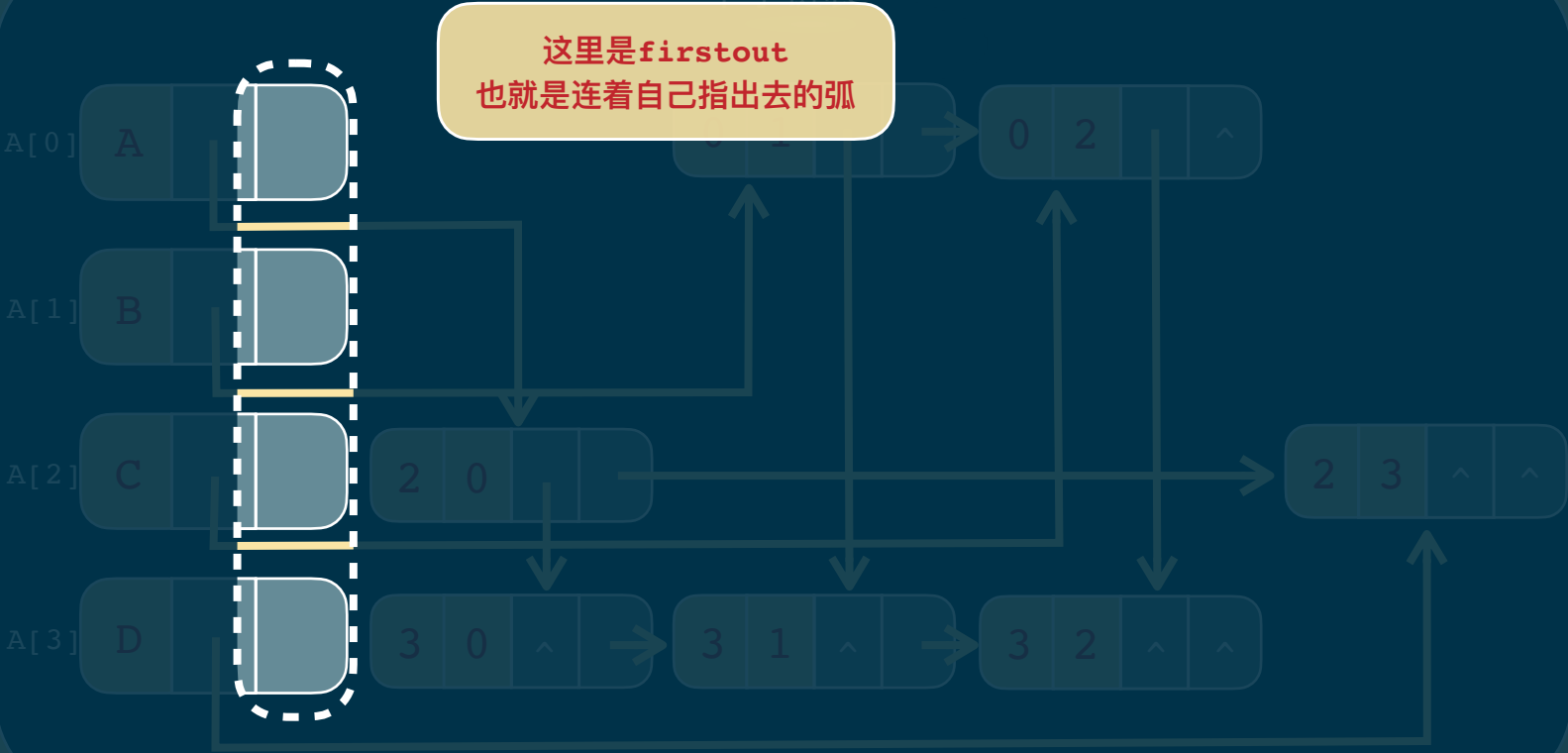
- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法

逻辑视角



十字链表

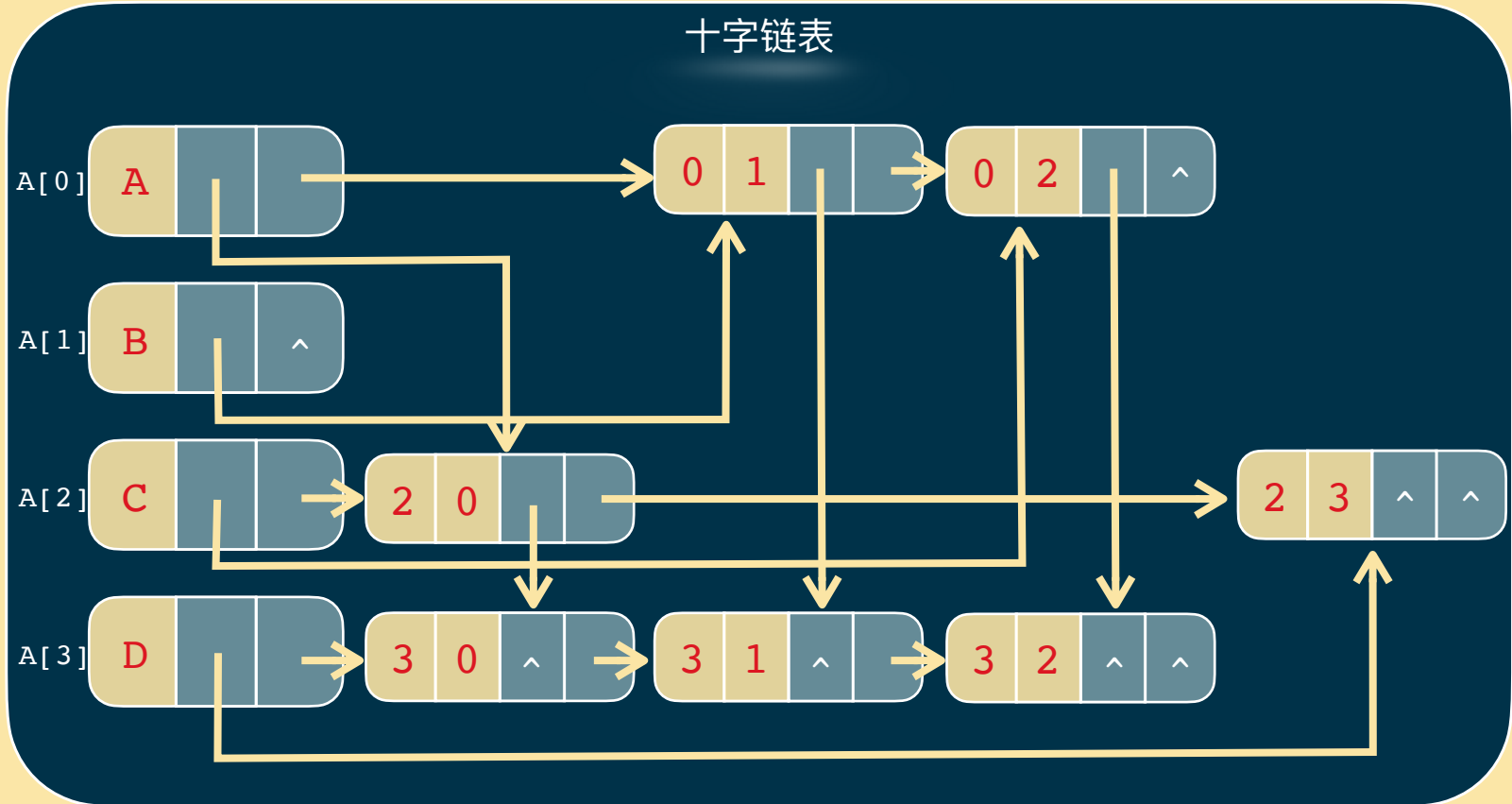
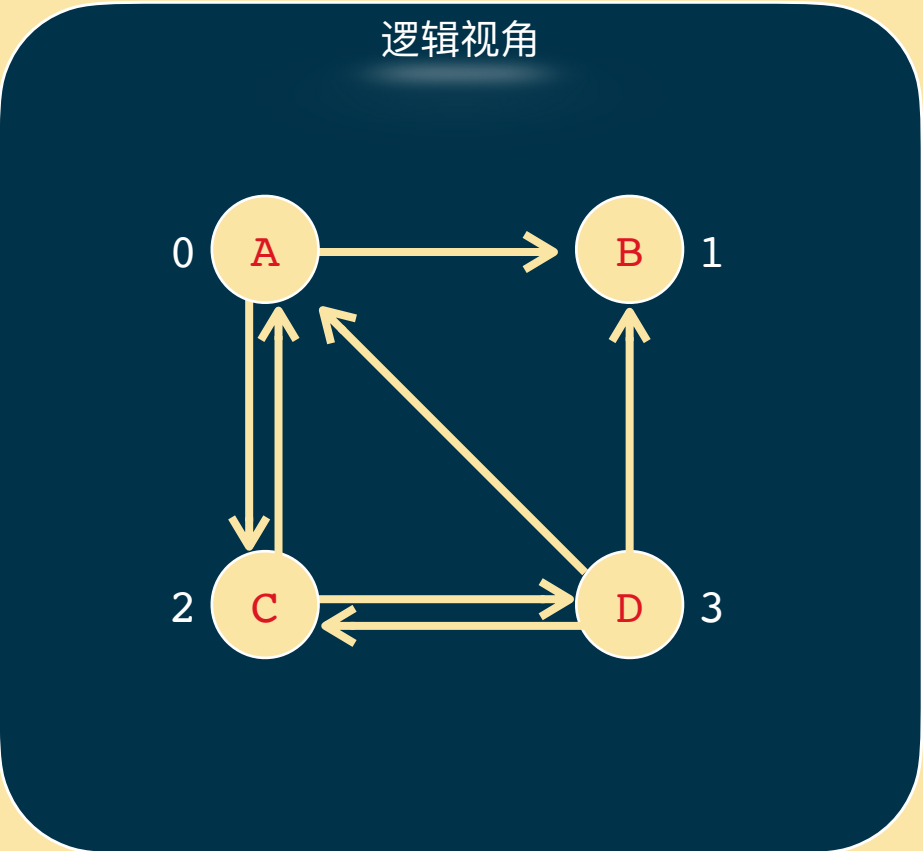




1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表**
(只适用有向图)
- 邻接多重表

十字链表的画法



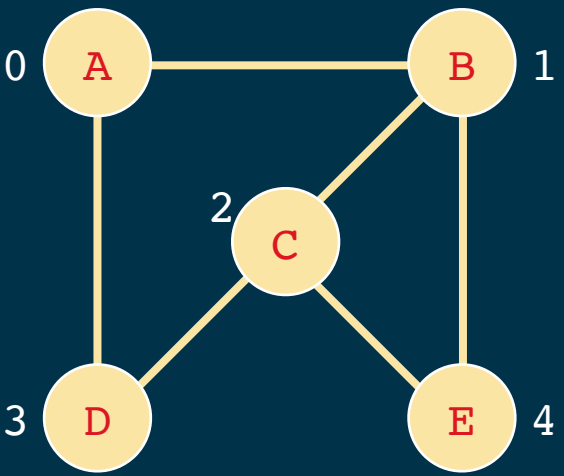
十字链表结合了邻接表和逆邻接表，可以方便地计算顶点 v_i 的入度和出度/寻找依附于某一结点的所有弧



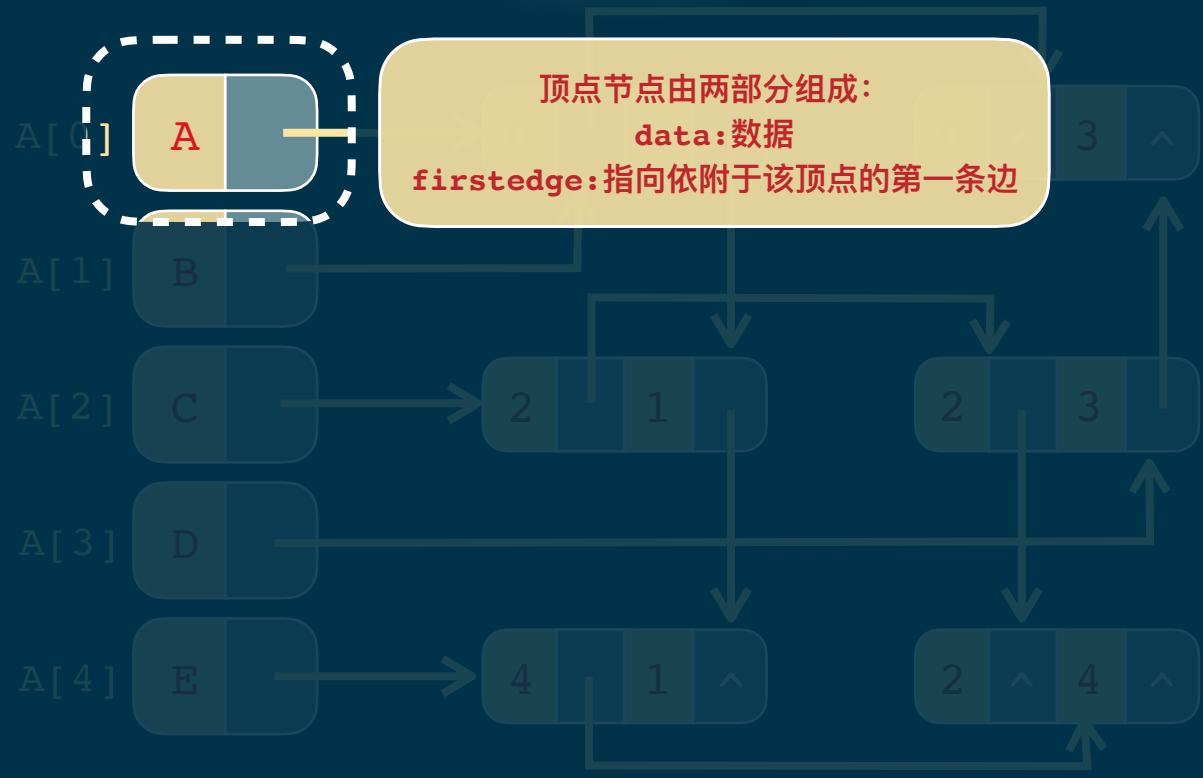
1. 图的存储结构

- 邻接矩阵法
- 邻接表法
- 十字链表
- 邻接多重表**
(只适用无向图)

逻辑视角



邻接多重表





1. 图的存储结构

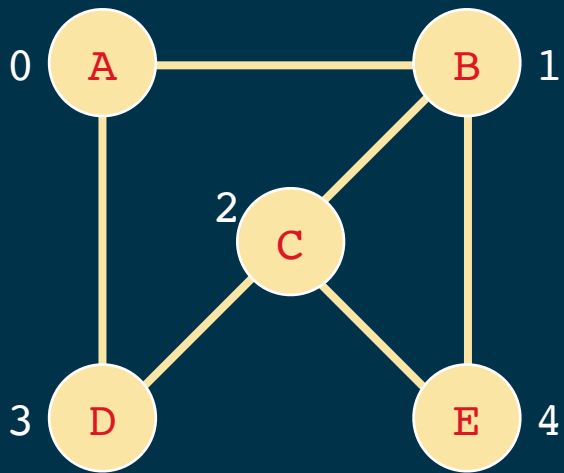
邻接矩阵法

邻接表法

十字链表

邻接多重表
(只适用无向图)

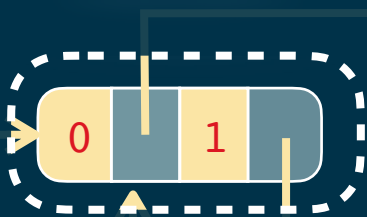
逻辑视角



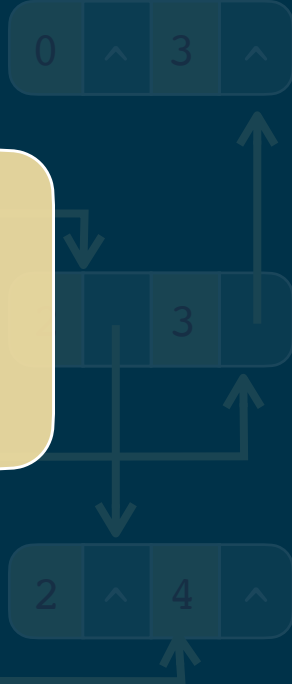
顶点表

A[0]	A
A[1]	B
A[2]	C
A[3]	D
A[4]	E

邻接多重表



边节点由5部分组成:
ivex:边依附第1个顶点的编号
ilink:依附于顶点i的下一条边
jvex:边依附第2个顶点的编号
jlink:依附于顶点j的下一条边
info:边的信息(如权重, 图中未画出)





1. 图的存储结构

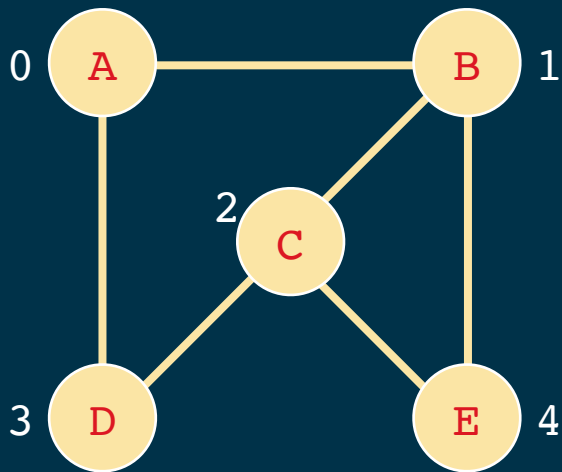
邻接矩阵法

邻接表法

十字链表

邻接多重表
(只适用无向图)

逻辑视角

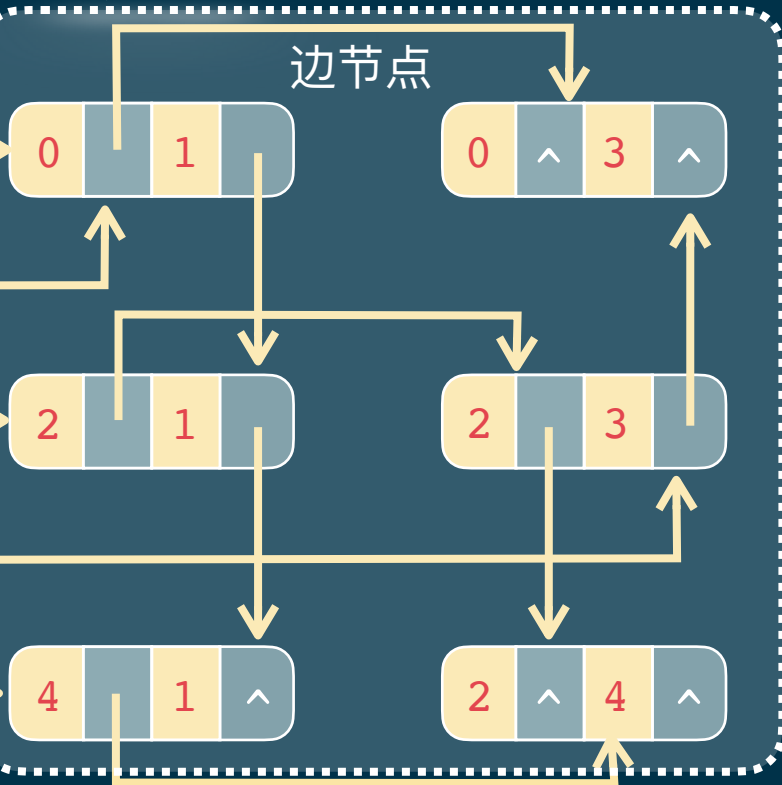


顶点表

A[0]	A	
A[1]	B	
A[2]	C	
A[3]	D	
A[4]	E	

邻接多重表

边节点



邻接多重表中，每条边只会被存储一次，且所有依附于同一结点的边都被串联在一个链表中，便于计算结点的度/寻找依附于某一结点的所有边



1. 图的存储结构

	邻接矩阵法	邻接表法	十字链表	邻接多重表
空间复杂度	$O(V ^2)$	无向图 $O(V + 2E)$ 有向图 $O(V + E)$	$O(V + E)$	$O(V + E)$
找相邻边	遍历对应行/列 复杂度 $O(V)$	有向图找入度 要遍历整个邻接表	方便	方便
删除边或点	删边方便 删点麻烦	无向图删边点都不方便	方便	方便
适用范围	稠密图	稀疏图	只能有向图	只能无向图
是否唯一	唯一	不唯一	不唯一	不唯一